Civic-Connect: Project Engineering Document (PED)

ENGINEERING LARPERS

Commented [TM1]: I changed because of updates needed

Table of Contents

Documentation: 5

Final Submission Information Block 5

Problem and Business Need Problem Statement 6

- Business Need 6

Stakeholder Analysis 6

- Stakeholder Conflict / Competing Expectations 7

Scope Baseline 7

- In-Scope 7
- Out-of-Scope 8
- Deferred / Future Scope 8

Requirements and Acceptance Criteria 9

- Functional Requirement 9
- Non-Functional Requirement 10
  - NFR-001: Response Time (Performance) 10
  - NFR-002: Concurrent Users (Scalability) 11
  - NFR-003: Data Encryption (Security) 11
  - NFR-004: Audit Logging (Accountability) 12
  - NFR-005: Availability (Reliability) 12
  - NFR-006: Accessibility (Usability) 13

Constraints 13

- Scope Constraints 13
- Schedule Constraints 14
- Cost / Resource Constraints 14
- Quality Constraints 15
- Security Constraints 15

Architecturally Significant Requirements (ASRs) & Quality Drivers 15

Architecture Decision 16
Alternatives Considered 16
Selected Architecture 17
B. Modular Monolith: 17
Architecture Diagram(s) 18
Architectural Decision Record 18
Data and Persistence Baseline 18
Key Entities 18
Initial Data Model / Schema 18
Persistence Technology Justification 18
Data/Persistence ADR — ADR-D01 19
Technology-Stack Decision 20
Comparison Evidence 20
Technology ADR — ADR-T01 21
Initial Design Decisions (Informed by A2 Research) 21
Design Problem 1: Status transition rules 21
Design Problem 2: Direct Instantiation of Notification Channel Classes 22
Design Problem 3: Data access decoupling 22
Initial API / Integration Decisions 23
API Decision 23
Deployment Compatibility 23
Architecture, Technology & Initial Design Baseline — Sign-Off 23
Meaningful Development Evidence 23
Initial Requirements Traceability Matrix 24
NFR Traceability 24
Constraint Traceability 24
Initial Risk Register 25

New M2 Risks Register 26
Forward Engineering Consideration 26
Engineering Decision Log 27
GitHub and Team Governance 27
AI Usage Register 35
PED Baseline Sign-Off 37

Documentation:

Version	Date	Purpose	Author/Reviewer
1.0	9 Sep 2026	Initial M1 baseline: problem, stakeholders, scope, requirements, constraints, risk, governance, sign-off	Engineering Larpers
2.0	29 Sep 2026	M2 evolution: architecturally significant requirements, architecture/technology/data decisions, initial design-pattern decisions informed by A2 research, updated RTM/risk/decision log, Architecture, Technology & Initial Design Baseline sign-off, initial application development evidence.	Engineering Larpers
2.1	30 Sep 2026	M2 evolution Updates	Engineering Larpers
3.0	16 Oct 2026	M3: construction, CI, test design, controlled changes, release readiness	Engineering Larpers
4.0	**19 Oct 2026**	M4 closure: final product, scope-to-delivery conclusion, final RTM, M3 closure, go-live decision, reflection	Engineering Larpers

Commented [TM2]: This is where we update the versions of the PED and its write offs

Commented [TM3]: Here is the updated PED 4.0

Final Submission Information Block

Field	Content
Team	Engineering Larpers
Members	Busisiwe **CONFIRM** [surname, student no.]; Katlego Lethole **CONFIRM** [student no.]; Tshegofatso Mashego **CONFIRM** [student no.]
GitHub repository	https://github.com/TshegoMashegoBC/SEN381-CivicConnect-
Final commit / tag	**CONFIRM** [e.g. v4.0-final and full commit hash; must be the exact version demonstrated]
PED version	v4.0
Final RTM location	Section 7 of this PED **CONFIRM** [plus repo path if kept as a controlled file]
Final product access	Local run per README (npm run dev: API :5000, client :5173) **CONFIRM** [add staging URL if one exists]

Field	Content
CI / test evidence	**CONFIRM** [GitHub Actions run links, test report]
Date submitted	**CONFIRM**

Problem and Business Need

Problem Statement

CivicConnect currently manages community service requests through a fragmented mix of channels — email, telephone, WhatsApp messages, spreadsheets and paper records. Because no single controlled record exists, requests are frequently duplicated, overlooked, incorrectly assigned or lost between channels. Requesters have little visibility into whether a request has been received, assigned, delayed, resolved or closed, which erodes trust in the organisation's service delivery. Staff struggle to prioritise work, establish clear ownership of individual requests, and coordinate effort across the team, while accountability for status changes and actions taken is weak because there is no single traceable history per request. Management, in turn, lacks reliable, consolidated information on outstanding, overdue and resolved work, making reporting manual, inconsistent and difficult to audit.

Business Need

CivicConnect needs a controlled digital platform that gives requesters, staff and management a single, reliable and traceable way to submit, manage, monitor and report on service requests — improving visibility and accountability across the request lifecycle without introducing an unsustainable technical, operational or financial burden on the organisation.

Stakeholder Analysis

Three primary stakeholder groups were identified from the CivicConnect scenario. Their needs, and their relative influence and interest in the platform, are summarised below.

Stakeholder	Main Needs	Influence / Interest
Requesters	Submit service requests; track status; view request history; receive meaningful, timely feedback when a request is accepted, updated or completed.	Medium influence / High interest

Staff	View and search relevant requests; assign or accept responsibility; update status through controlled transitions; record actions/comments; resolve or close requests efficiently.	High influence / High interest
Management	Monitor outstanding, overdue, resolved and closed requests; view activity by category/status; access enough information to support accountability and service-performance reporting.	High influence / High interest

Stakeholder Conflict / Competing Expectations

The clearest tension identified is between requester transparency and staff workload control. Requesters want frequent, granular visibility into their request (who it is assigned to, why it is delayed, what is being done), because this directly drives their trust in the service. Staff, however, need the freedom to triage, batch and prioritise work without being interrupted by constant status queries, and are wary of a system that exposes internal workload or assignment decisions in a way that invites pressure from requesters or management.

This conflict was resolved for the M1 baseline by scoping requester-facing status visibility to a small set of controlled, staff-driven status values (e.g. Submitted → Assigned → In Progress → Resolved → Closed) rather than free-text or real-time internal detail. This satisfies the requester's core need for visibility (FR-002) while preserving staff control over how and when status changes are communicated (FR-005) and is reflected as a deliberate scope decision the team can defend at the M1 baseline sign-off.

Scope Baseline

In-Scope

All FRs identified are committed to the M1 baseline:

ID	Capability	FR Link	Priority
S-001	Request Submission	FR-001	High
S-002	Status Tracking	FR-002	High
S-003	Staff Request Queue	FR-003	High
S-004	Request Assignment	FR-004	High
S-005	Status Transitions	FR-005	High

S-006	Action/Comments Logging	FR-006	Medium
S-007	Management Monitoring	FR-007	Medium

Out-of-Scope

ID	Exclusion	Rationale	Stakeholder Impact
OS-001	Mobile Application (Native)	FRs assume web-based interaction, mobile would require separate codebases (iOS/Android), testing overhead, and ongoing operational costs. The team lacks expertise in mobile development.	Acceptable - responsive web application meets 80% of user needs for M1. Management approved web-only for Phase 1.
OS-002	Email/WhatsApp Integration	Not covered in FRs. Would require complex API integration, message parsing, and authentication challenges with significant security and reliability risks.	Acceptable - requesters will be onboarded to the web platform.
OS-003	Real-time Notifications (SMS/Push)	FRs do not require push notifications. Real-time notifications require third-party SMS gateways or push services with ongoing operational costs not approved in the budget.	Acceptable - feedback provided via web interface e and email updates only.
OS-004	Advanced Analytics / BI Reports	FR-007 only requires basic operational metrics. Complex reporting and data visualisation tools are outside current scope.	Acceptable - management accepts basic dashboard for M1, with advanced reporting deferred.
OS-005	Multi-language Support	Not identified as a stakeholder need in stakeholder analysis. Multi-language support would add significant UI, validation, and error messaging complexity without immediate stakeholder value.	Acceptable - staff confirmed English-only is acceptable for initial deployment.

Commented [TM4]: Exclude out of scope

Commented [TM5]: Put into either in scope or future scope

Deferred / Future Scope

ID	Deferred Item	Reason for Deferral
DS-001	Mobile Application	Requires separate platform development and user research.
DS-002	Email Integration (Inbound)	Allows time to evaluate email parsing libraries and security implications.
DS-003	SLA (Service Level Agreement) Reporting	Requires operational data to define meaningful SLAs.
DS-004	Automated Escalation Rules	Depends on actual usage patterns and staff workflows observed post-deployment.
DS-005	Bulk Request Import (CSV/Excel)	Staff capability to batch-create requests for maintenance events.

Requirements and Acceptance Criteria

Functional Requirement

The functional requirements below are derived directly from the stakeholder needs identified in Section 2. Each requirement has a unique identifier, an explicit source stakeholder, an indicative priority, and a testable acceptance criterion, in line with the Master Project Brief's requirements standard (Section 11).

ID	Functional Requirement	Source	Priority	Acceptance Criterion
FR-001	The system shall allow requesters to submit a new service request with the required information.	Requester	High	A requester can submit a request, and the system creates a unique request record.
FR-002	The system shall allow requesters to view the current status of their submitted requests.	Requester	High	A requester can select a submitted request and see its current, controlled status.
FR-003	The system shall allow authorised staff to view service requests relevant to them.	Staff	High	An authorised staff member can access their relevant requests after logging in.
FR-004	The system shall allow authorised staff to assign or accept responsibility for a request.	Staff	High	A staff member can assign/accept a request, and the responsible person is recorded.
FR-005	The system shall allow authorised staff to update the status of a request through controlled status transitions.	Staff	High	A permitted status change updates the request and records the new status and timestamp.

FR-006	The system shall allow staff to record comments, actions or resolution information against a request.	Staff	Medium	A staff member can add information to a request, and it is saved against that request, retaining its author and time.
FR-007	The system shall allow management to identify open, overdue, resolved and closed requests.	Management	Medium	Management can view requests filtered or grouped by these statuses.

Non-Functional Requirement

Overview
All NFRs are derived from stakeholder needs and the project's constraints identified in the Constraints Analysis document. Each NFR is **measurable** - written with specific targets and includes acceptance criteria.

NFR-001: Response Time (Performance)

Attribute	Detail
ID	NFR-001
Description	The requester dashboard must load within 2 seconds when accessed by a user on astandard broadband connection.
Source	Requester needs: "Submit requests, track status, view history" requesters need a responsive system.
Priority	High
Measurement	Use Chrome DevTools Network tab to measure Time to First Byte (TTFB) and full-page load time. Test with 3 representative pages: Dashboard, Request List, and Request Detail. Run 10 consecutive measurements.
Acceptance Criteria	AC-NFR-001: Dashboard page loads in < 2 seconds in 95% of test runs (10 consecutive measurements).
How It Constrains M2 Architecture	Forces the team to consider caching database indexing, efficient API design (avoid N+1 queries), and server-side rendering vs. client-side rendering trade-offs. Technology choice must support these patterns.

Downstream Risk	If the selected frontend framework is heavy, or the database is not indexed, performance will fail. Requires performance testing in M3. If free-tier database performance is insufficient, may need to increase threshold to 3 seconds or implement more aggressive caching.

NFR-002: Concurrent Users (Scalability)

Attribute	Detail
ID	NFR-002
Description	The system must support at least 50 concurrent users without significant degradation. "Significant degradation" defined as response time > 4 seconds on the dashboard.
Source	Staff and Management needs stakeholder analysis: "Multiple staff working simultaneously" - the system must handle real-world usage.
Priority	Medium
Measurement	Use Apache JMeter or k6 to simulate 50 concurrent users performing typical actions (view dashboard, submit request, update status) over a 5-minute period.
Acceptance Criteria	AC-NFR-002: All requests complete successfully (HTTP 200) with average response time < 3 seconds and maximum < 5 seconds.
How It Constrains M2 Architecture	Forces the team to consider database connection pooling, stateless application design for horizontal scaling, and load balancing. Technology must support multiple concurrent connections (e.g., Node.js event loop, or Java thread pooling).
Downstream Risk	If the team chooses a framework with poor concurrency support (e.g., PHP without Apache tuning), or a database with low connection limits (e.g., free-tier PostgreSQL with 20 connections), this NFR will fail. Free-tier limitations are a key concern (see Trade-Off 2 in Constraints Analysis).

NFR-003: Data Encryption (Security)

Attribute	Detail
ID	NFR-003
Description	All data transmitted between the client and server must be encrypted using TLS 1.2 or higher.
Source	Security constraint from Master Project Brief (SE-001): "Security is a lifecycle-wide responsibility" and SE-003: "Data in transit must be encrypted."
Priority	High

Measurement	Use SSL Labs SSL Server Test (or equivalent tool) to verify the deployed site uses TLS 1.2+ and has no known vulnerabilities (e.g., POODLE, Heartbleed).
Acceptance Criteria	AC-NFR-003: SSL Labs test returns an overall rating of "A" or "A+".
How It Constrains M2 Architecture	Forces the team to choose a deployment platform that provides free SSL certificates. Cannot use plain HTTP. Must enforce HTTPS redirection at the application or server level.
Downstream Risk	If the selected platform doesn't support automatic SSL (e.g., some free hosting services), the team must manually configure certificates, creating an operational burden.

NFR-004: Audit Logging (Accountability)

Attribute	Detail
ID	NFR-004
Description	All status changes on service requests must be logged with: timestamp, user ID, previous status, new status, and optional comment.
Source	Staff and Management needs stakeholder analysis. "Accountability" and the Master Project Brief's problem statement about "weak accountability for changes to request status."
Priority	High
Measurement	Manually verify audit log entries in the database after performing status transitions. Query the audit table to check completeness of fields.
Acceptance Criteria	AC-NFR-004: After 5 distinct status transitions, the audit log contains 5 entries with all required fields populated (timestamp, user ID, previous status, new status).
How It Constrains M2 Architecture	Forces the team to design the database schema to support an audit log table. The architecture must ensure that every status update triggers an audit record - this may be implemented via database triggers, ORM hooks, or explicit service-layer logic. Technology choice must support database transactions to ensure atomicity (status update and audit log must succeed or fail together).
Downstream Risk	If the team forgets to implement audit logging early, retrofitting it later will be difficult and error prone. Must be designed in M2 and implemented in M3.

NFR-005: Availability (Reliability)

Attribute	Detail
ID	NFR-005
Description	The system must be available 99.5% of the time during business hours.
Source	Staff and Management needs stakeholder analysis. "Monitor and analyse performance" - you can't monitor or analyse if the system is down.
Priority	Medium

Measurement	Monitor uptime using UptimeRobot or similar tool over a 2-week period during business hours.
Acceptance Criteria	AC-NFR-005: Uptime >= 99.5% (maximum 43 minutes of downtime per month during business hours) over a 2-week test period.
How It Constrains M2 Architecture	Forces the team to consider deployment platform reliability (choose a reputable cloud provider with good SLAs), error handling (graceful failure), database backup/recovery, and health check endpoints for monitoring. Technology should support hot-reload or zero-downtime deployment if possible.
Downstream Risk	Free-tier platforms may have less reliability (e.g., Heroku dyno sleeps after 30 minutes idle). The team may need to choose a platform with better uptime guarantees, potentially incurring cost (see Trade-Off 2 in Constraints Analysis).

NFR-006: Accessibility (Usability)

Attribute	Detail
ID	NFR-006
Description	The web interface must meet basic WCAG 2.1 Level A accessibility standards.
Source	Organisation's inclusivity policy (assumed/implicit quality expectation) and good engineering practice.
Priority	Low
Measurement	Use automated accessibility checker (e.g., axe DevTools, Lighthouse) to scan all major pages: Dashboard, Request Form, Request List, and Request Detail pages.
Acceptance Criteria	AC-NFR-006: No Level A violations on Dashboard, Request Form, Request List, and Request Detail pages as reported by axe DevTools.
How It Constrains M2 Architecture	Forces the team to choose a frontend framework with good accessibility support (e.g., semantic HTML, ARIA attributes). Design decisions must consider colour contrast, keyboard navigation, and screen reader support.
Downstream Risk	If the team uses a UI library that doesn't prioritise accessibility (e.g., some custom component libraries), they'll need to manually add ARIA roles, increasing development time.

Constraints

Scope Constraints

ID	Constraint	Description	Engineering Implication
SC-001	Fixed Scope	All 7 FRs (FR-001 to FR-007) identified are committed to the baseline.	Cannot drop any FRs; any scope change requires formal change control with impact analysis.
SC-002	Minimum Capabilities	The system must support all Minimum Business Capabilities defined in the Master Project Brief.	Cannot drop core features; prioritisation applies to additional features only.

SC-003	No Unapproved Additions	Additional features are permitted only if justified by stakeholder value and effect on scope/schedule/quality/security/risk is considered.	Each proposed new FR requires impact analysis; "nice-to-have" features are deferred to DS-001 to DS-005.

Schedule Constraints

ID	Constraint	Description	Engineering Implication
SH-001	4 Milestones	Four formal milestones within the SEN381 delivery period: M1 (Week 4), M2 (Week 8), M3 (Week 12), M4 (Week 16).	Cannot delay M1 without affecting downstream phases. All 7 FRs and 6 NFRs must be deliverable within the fixed timeline.
SH-002	Fixed End Date	No extension beyond academic calendar.	Scope must be realistic; defer non-essential features (DS-001 to DS-005) to maintain quality.
SH-003	Review Lead Time	Two-reviewer approval required for substantive changes entering main.	Team must plan for review cycles; cannot rush changes at the last minute.

Cost / Resource Constraints

ID	Constraint	Description	Engineering Implication
CR-001	Free/Low-Cost Services	Prefer free or low-cost services where practical; identify limitations.	Technology choices must evaluate free-tier limitations like deployment platform free tier, database size limits, connection limits. Must support all 7 FRs and 6 NFRs on free tier.
CR-002	Team Size	Exactly 3 students; no additional paid developers.	Work must be divided efficiently; complexity must match team capacity.
CR-003	Operational Cost	Likely operational cost beyond educational context must be identified.	Hosting, domain, third-party API costs must be documented and reasonable.
CR-004	Learning Curve	Unfamiliar technologies introduce schedule risk.	Technology selection must consider team's existing skills; if adopting new tech, factor in learning time.

Quality Constraints

ID	Constraint	Description	Engineering Implication
QU-001	Measurable Quality	Quality attributes must be defined and supported by measurable evidence	All NFRs (NFR-001 to NFR-006) are written with measurable targets.
QU-002	Acceptance Criteria	Important requirements must have acceptance criteria.	Every FR and every NFR has defined acceptance criteria.
QU-003	Test Evidence	Test coverage alone is insufficient; evidence must show what was verified and what was not.	Unit tests, integration tests, and manual tests must be traceable to requirements in the RTM.
QU-004	Defect Management	Defects must be recorded with severity/priority and disposition.	Cannot ignore bugs; must track and address them.

Security Constraints

ID	Constraint	Description	Engineering Implication
SE-001	Lifecycle Responsibility	Security is a lifecycle-wide responsibility, not a final add-on.	NFR-003 (encryption) and NFR-004 (audit logging) defined in M1; security must be considered in M2 architecture, M3 implementation, and M4 deployment.
SE-002	Secrets Handling	Passwords, API keys, tokens, private keys must not be committed to GitHub.	Must use environment variables and .gitignore; enforce with GitHub pre-commit hooks.
SE-003	Data in Transit	All sensitive data (requests, user info) must be encrypted in transit (TLS 1.2+).	Technology choice must support HTTPS; deployment platform must provide SSL certificates (NFR-003).
SE-004	Authentication & RBAC	Authentication and Role-Based Access Control are mandatory.	Cannot skip authentication for simplicity; must design RBAC from the start (supports FR-003, FR-004, FR-007).
SE-005	Residual Risks	Cannot claim system is "secure"; record residual security risks.	Threat modelling must identify weaknesses; risk register must document accepted risks.

Architecturally Significant Requirements (ASRs) & Quality Drivers

ASR ID	Requirement/Quality Driver	Why It Drives Architecture	Measurable Expectation	Linked Evidence
ASR-001	NFR-004 — Audit logging / accountability	Every status transition must be atomically recorded; affects data-layer and transaction design.	AC-NFR-004	R005, D-00X

ASR-002	NFR-002 — Concurrency (50 users)	Drives connection-pooling, statelessness and persistence choice.	AC-NFR-002	R001, R003
ASR-003	SE-004 — Authentication & RBAC	Drives module boundaries (requester/staff/management) and trust boundary placement.	—	R005
ASR-004	NFR — Scalability	This has to do with whether we need to scale up or down horizontally/vertically in order to improve for more users and more functions	—	R006
ASR-005	NFR-005 — Availability	This drives for more concurrent users to be used, if whether it will also be scaled up or down for access	—	R007
ASR-006	SE-004 — Authentication & RBAC	This is for security measures and access control of information and measures to improve management	—	R005

Architecture Decision

Alternatives Considered

Option	Description	Rejected? Why / Why Not
A	Layered monolith	Rejected,
B	Modular monolith with clear module boundaries	Accepted
C	Microservices	Rejected
D	SOA	Rejected

Selected Architecture

B. Modular Monolith:

A modular monolith remains one principal deployable application. It may run as one instance or several equivalent instances behind a load balancer.
When the application scales horizontally, every instance normally contains all modules. This is operationally simpler than many independent services, but selective module scaling is limited.

- Usability & traceability for requesters/staff/management → satisfied by a single SPA with role-based views, not three separate front-ends.
- Maintainability with a small team → one repo, one deployable unit, one CI pipeline to maintain.
- Security/RBAC → centralised in the API layer (JWT + role claims), rather than duplicated across services.
- Auditability of status changes → a dedicated StatusHistory concern inside the business logic layer, not a distributed event log.
- Deployability on a free/low-cost tier → one API + one static SPA deploys comfortably within Render/Azure/Fly.io free tiers; a services-per-container design would multiply cost and DevOps effort for no requirement-driven benefit.

Business Capabilities:

Capability	FRs	ASRs That Shape It
Request Management	FR-001, FR-002, FR-005	NFR-004 (audit), NFR-001 (performance)
Staff Assignment	FR-003, FR-004, FR-006	FR-004/005 (RBAC, state machine), NFR-004
Reporting/Oversight	FR-007	NFR-002 (concurrency), NFR-005 (availability)
Identity & Access	FR-001-FR-007	SE-004 (RBAC), NFR-003 (TLS)
Audit & Compliance	FR-001-FR-007	NFR-004 (audit logging)

Module Boundaries:

Module	Owns	Justification
Identity & Access	Users, roles, sessions	SE-004 requires RBAC; authentication is a distinct concern
Request Management	Requests, statuses, categories	Core business capability; FR-001/002/005
Staff Assignment	Assignments, comments, actions	FR-003/004/006; distinct workflow from request creation
Reporting	Read models, aggregates	FR-007; read-heavy, different performance profile (NFR-002)
Audit	Audit log entries	NFR-004; cross-cutting, append-only, compliance-critical

Architectural Constrains

Constraint ID	Description	ASR Impact
SC-001	Fixed scope (all 7 FRs committed)	Cannot drop FRs; ASRs must be satisfied within fixed scope
SH-001	4 milestones, fixed dates	ASR-driven decisions must be made by M2; cannot delay architecture
CR-001	Free/low-cost services preferred	NFR-003 (SSL) and NFR-005 (availability) constrained by free-tier limits
CR-002	Team of exactly 3 students	Complexity of ASR-driven architecture must match team capacity

SE-001	Security lifecycle-wide	NFR-003 and NFR-004 are not optional add-ons
SE-004	Authentication & RBAC mandatory	FR-003/FR-004/FR-007 require RBAC design from the start

Architecture Diagram(s)

Please reference to our GitHub Repo.

Architectural Decision Record

Please reference to our GitHub Repo

Data and Persistence Baseline

Key Entities

Please reference to our GitHub Repo

Initial Data Model / Schema

Please references to our GitHub Repo

Persistence Technology Justification

Please reference to our GitHub Repo

Criterion	Assessment
Structure	Entities have fixed, well-understood relationships (one request has many status-history rows and comments, and references one requester, one optional assignee, one category) — a relational schema expresses this directly through foreign keys rather than through application-managed references
Access patterns	Staff need to search/filter/sort requests by status, category and assignee (FR-003) and management need aggregate counts by status/category (FR-007); both are naturally expressed as indexed relational queries and joins

Integrity / consistency	NFR-004 requires that a status change and its audit-log entry are never recorded separately — a relational database's ACID transactions let the status_requests.status update and the status_history insert commit or fail together, which a weakly consistent store cannot guarantee without extra application logic
Sensitivity	Request data includes personal contact details and, potentially, sensitive complaint content; a relational schema with explicit foreign keys and constraints makes it easier to reason about and restrict who can read what, supporting SE-004/NFR-003
Expected growth	CivicConnect's data volume (a community organisation's service requests) is modest and the entity count is small and stable; a relational model does not need to anticipate schema-less growth the way a document store would for unpredictable record shapes

Data/Persistence ADR — ADR-D01

Field	Content
Context	PostgreSQL choice of Database
Constraints	Reference to Selected Decision
Alternatives	Reference to Selected Decision
Decision	Modular Monolith
Rationale	[why it fits CivicConnect specifically]
Trade-offs	Reference to Selected Decision
Risks	[]
Evidence	[Week 2 teaching reference / A2 section, if applicable]
Later consequence	To be updated at M3/M4 once implementation evidence exists.

Technology-Stack Decision

Comparison Evidence

Criterion	Option 1: Django + HTMX + Tailwind + PostgreSQL	Option 2: ASP.NET Core Web API + React (Vite/TS) + PostgreSQL	Option 3: Node.js (NestJS/Express) + React/Next.js + PostgreSQL
Requirement/ASR fit	Django's built-in auth, Groups/Permissions, ORM and admin cover RBAC, auditability (django-simple-history or a custom AuditLog) and reporting screens with little custom code. State-pattern workflow via django-fsm.	The API layer centralises RBAC (Identity + JWT role claims). Strategy pattern suits status transitions, and Repository/Unit of Work gives atomic status-plus-history writes. Optimistic concurrency via xmin/row version is easy to add.	Covers the functional needs (RBAC, workflow, reporting), but auth, RBAC and audit must be assembled from libraries. Less is built in for security and auditability.
Team capability / learning curve (CR-004)	One language (Python) and no separate SPA or REST layer, so the learning curve is low. It is best if the team knows Python. HTMX and django-fsm are new to most students.	Builds on prior C#/.NET coursework (OOP, algorithms) and full-stack JS experience. The team has two languages and two tooling chains (.NET and React/TS).	Fits a team that is already strongest in JS/TS, and it keeps a single language across front and back end. Two frameworks (NestJS plus React/Next) and more manual setup.
Cost / licensing (CR-001, CR-003)	All open source with no licence fees. Free tiers: Render/Railway (hosting), Neon/Supabase (PostgreSQL), Sentry. Free-tier limits are a recorded risk (R-006).	.NET, EF Core, React and PostgreSQL are all free. The team can use Render or Azure for Students credits. No external auth cost, since Identity + JWT runs in-app.	All open source and free. Uses the same free-tier hosting and PostgreSQL options. Auth0/Firebase would add cost or dependency, so self-managed auth is preferable.
Security ecosystem	Strong out of the box: built-in CSRF/XSS/SQL-injection protection, session auth, password hashing and migrations. Tooling: pip-audit, Bandit, Dependabot.	Strong: ASP.NET Core Identity, built-in DI, model validation and JWT auth. Dependabot and GitHub Actions scans are available. JWT storage and handling on the SPA needs care.	Mature libraries (Passport, Helmet, class-validator), but security is more manual, and dependency sprawl in npm raises supply-chain risk. Dependabot and npm audit help.
Deployment compatibility	Simple: one web service plus one database on Render/Railway with Neon/Supabase. Static files via WhiteNoise. Matches the modular-monolith deployment view.	One API plus one static SPA deploys within Render, Azure or Fly.io free tiers. Needs two deployables (API and SPA) and CORS configuration.	API and SPA deploy to Render, Vercel or Netlify. Also two deployables, with the same CORS and environment configuration.
Maintainability	One codebase and one framework, with strong conventions. Django's batteries-included approach reduces custom code. Risks: HTMX-heavy UIs can become hard to structure, and Django signals can hide control flow.	Clear layering (Presentation, API, Business, Data) and strong typing on both sides. Repository/UoW supports fast unit tests without a live database. More abstraction and boilerplate for a small domain.	TypeScript end to end helps maintainability, and the tooling is familiar. More moving parts and fewer enforced conventions, so a small team needs more discipline to stay consistent.

Technology ADR — ADR-T01 Intial

Field	Content
Context	Option 2
Alternatives	Reference Technology Stack Decision
Decision	[final frontend / backend / persistence / tooling stack]
Rationale	Reference Technology Stack Decision
Trade-offs / Risks	[link R001, R003]

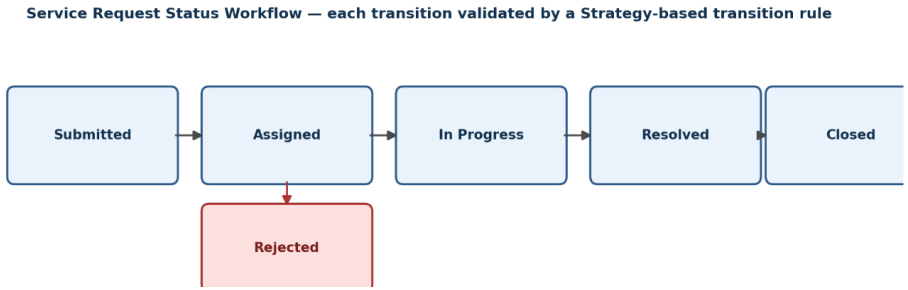
Technology ADR — ADR-T02 Final

Field	Content
Context	Option 2
Alternatives	Reference Technology Stack Decision
Decision	[final frontend / backend / persistence / tooling stack]
Rationale	Reference Technology Stack Decision
Trade-offs / Risks	[link R001, R003]

Initial Design Decisions (Informed by A2 Research)

Design Problem 1: Status transition rules

Problem: a service request moves through Submitted → Assigned → In Progress → Resolved → Closed (or Rejected), and each transition has different validation rules (who may perform it, what data is required, what notification fires). Hard-coding this as nested if/else in a controller becomes unreadable and untestable as rules grow.



- Pattern selected: Strategy — one ITransitionRule implementation per status change, injected into a StatusWorkflowService.
- Benefit: each rule is independently unit-testable; adding a new transition (e.g. “Reopen”) means adding one class, not editing a large conditional.

Trade-off/complexity introduced: more classes/files for a small state machine; justified because status-transition correctness is a core, frequently-changed business rule in CivicConnect.

Design Problem 2: Direct Instantiation of Notification Channel Classes

The Problem (before naming a solution). When a request changes status, the part of the code that decides a requester or staff member should be notified currently constructs the concrete notification class directly (for example, calling the email-channel class's constructor, or the SMS-channel class's constructor, depending on a stored contact-preference value). Every place in the codebase that needs to send a notification repeats this same conditional construction logic, and every place must be edited if CivicConnect adds a new channel (for example, in-app notifications) or changes how an existing channel is configured.

Design Problem 3: Data access decoupling

Problem: controllers and services need to query/persist Users, ServiceRequests, Categories etc. without being tightly bound to EF Core, so the domain logic stays testable and the ORM could be swapped later without a rewrite.

- Pattern selected: Repository + Unit of Work — one repository interface per aggregate (IServiceRequestRepository, IUserRepository), coordinated by a single IUnitOfWork that commits a transaction per business operation.
- Benefit: business logic can be unit-tested against in-memory fakes; a status-change-plus-history-write happens inside one atomic commit.
- Trade-off: an extra abstraction layer over EF Core's own repository-like DbSet/DbContext — justified because it keeps service-layer tests fast and decoupled from a live database, which matters for a team with limited shared testing time.

Initial API / Integration Decisions

API Decision

Please reference to our GitHub Repo

Deployment Compatibility

Please future reference to our GitHub Repo

Architecture, Technology & Initial Design Baseline — Sign-Off

Field	Content
Baseline type	M2 — Architecture, Technology & Initial Design Baseline
Version	PED v2.0
Date	29 Sep 2026
Included in this baseline	Architecture (ADR-A01), persistence (ADR-D01), technology stack (ADR-T01), Design Problems 1 & 2, updated RTM/Risk Register
Known open decisions / deferred concerns	All
Scope reviewed	YES
Requirements/traceability checked	YES
Risk review completed	YES
Repository/governance controls checked	YES
Outcome	[REVISION REQUIRED]

Meaningful Development Evidence

Please reference to our Github Repo

Initial Requirements Traceability Matrix

NFR Traceability

NFR ID	Source	Acceptance Criteria	ASR Link	Design Link (M2)	Test Link (M3)
NFR-001	Requester	AC-NFR-001: Dashboard loads in < 2s (95% of tests)			TBD
NFR-002	Staff/Management	AC-NFR-002: 50 concurrent users, avg < 3s, max < 5s			TBD
NFR-003	Security Constraint	AC-NFR-003: SSL Labs rating A/A+			TBD
NFR-004	Staff/Management	AC-NFR-004: All status changes logged with required fields			TBD
NFR-005	Staff/Management	AC-NFR-005: 99.5% uptime during business hours			TBD
NFR-006	Org Policy (implicit)	AC-NFR-006: No WCAG 2.1 Level A violations			TBD

Constraint Traceability

Constraint ID	Source	Engineering Implication	M2 Impact	ASR Link	M3 Impact
SC-001	Master Project Brief	Fixed scope; change control required	Design decisions must be reversible/adaptable		Changes require impact analysis
SC-002	Master Project Brief	Minimum capabilities must be delivered	Architecture must support all FRs		All FRs must be implemented
SC-003	Master Project Brief	No unapproved additions	Each proposed feature requires justification		No unapproved scope creep
SH-001	Master Project Brief	4 milestones, fixed dates	Must deliver architecture by M2		Must deliver working code by M3
SH-002	Master Project Brief	Fixed end date	Realistic scope; defer non-essential features		Must complete by deadline
SH-003	Master Project Brief	2 approvals for PRs	Plan for review cycles		Cannot rush changes
CR-001	Master Project Brief	Free/low-cost services	Platform selection constrained by free-tier limits		Deployment platform must be free/compatible
CR-002	Master Project Brief	Team of 3	Complexity must match team capacity		Work must be divided efficiently
CR-003	Master Project Brief	Operational cost	Document projected costs		Include cost review in final
CR-004	Team Analysis	Learning curve	Factor learning time into planning		Monitor learning curve impact

QU-001	Master Project Brief	NFRs must be measurable	Architecture must support performance testing	Tests must validate NFRs
QU-002	Master Project Brief	Acceptance criteria	Design must meet ACs	Tests must verify ACs
QU-003	Master Project Brief	Test evidence	Architecture must support testability	Tests must be traceable to requirements
QU-004	Master Project Brief	Defect management	Track defects	Address defects systematically
SE-001	Master Project Brief	Security lifecycle-wide	Security patterns built into architecture	Security scanning in CI/CD
SE-002	Master Project Brief	Secrets handling	Environment variables in architecture	No secrets in GitHub
SE-003	Master Project Brief	Data in transit encryption	Platform must support HTTPS	TLS enforced
SE-004	Master Project Brief	Authentication & RBAC	Design RBAC from the start	Implement authentication
SE-005	Master Project Brief	Residual risks	Document accepted risks	Record residual security risks

Initial Risk Register

Commented [TM6]: @Tshegofatso Mashego Changing and adding new table to risk register

ID	Description	Cause	Prob.	Impact	Priority	Mitigation	Contingency	Owner	Status
R-001	Team lacks prior experience with the backend framework likely to be chosen in M2	Skills gap / learning curve	Medium	High	High	Timebox a technical spike in the first week of M2 before committing	Fall back to a framework the team already knows from prior modules	Busiswe & Tshegofatso	Open
R-002	Stakeholder (management) requests features beyond the agreed MVP scope	Enthusiastic but uncontrolled stakeholder input	Medium	Medium	Medium	Apply the Change Request/Impact Analysis template (Master Brief Appendix E) to any new ask	Log as deferred/future scope if it cannot be absorbed without schedule impact	Katlego	Open
R-003	Free-tier hosting/database limits are exceeded during load testing or demo	Under-provisioned free-tier resources	Low	Medium	Medium	Monitor usage against documented free-tier limits from M2 onward	Migrate to a low-cost paid tier or reduce load-test scale for the demo	Busiswe & Tshegofatso	Open
R-004	Two-reviewer approval is skipped under deadline pressure close to a milestone	Time pressure vs. governance discipline	Medium	Medium	Medium	Enforce branch protection technically in GitHub settings, not	Retrospective review of merge history before each milestone submission	Busiswe & Tshegofatso	Open

						just by agreement			
R-005	A baselined requirement is edited after sign-off without a formal change record	Informal communication overriding controlled process	Low	High	Medium	Team agreement: all baseline edits go through the Change Request template	Revert to last baselined version and reprocess the change formally	Requirements Engineer	Open

New M2 Risks Register

ID	Description	Cause	Prob.	Impact	Priority	Mitigation	Owner	Status
R006	Selected persistence technology's free-tier connection limit blocks concurrency target	By the inconsistents of which the business qualities don't align with the modules which don't align with	Medium	High	Mediun	TBD	Busisiwe	Open
R007	Chosen design pattern adds complexity the team has not implemented before	By the inconsistent of which the business qualities don't align with the modules which don't align with	Medium	High	Mediun	TBD	Everyone	Open

Forward Engineering Consideration

Commented [TM7]: Please reconsider and change @Tshegofatso Mashego

Concern	Why it matters now	Later decision/activity it influences	Evidence still missing	Risk of ignoring it
Security (RBAC & data protection)	Shapes the data model and authentication requirements before they're designed.	Authentication mechanism/technology choice in M2.	Which specific permissions each of the 3 roles requires, in detail.	Bolted-on security in M3 causes rework and weakens the security-evidence marks.
Testability	NFRs must already be written so they can be tested later, not rewritten in M3.	Test framework/tooling choice, aligned to the M2 technology decision.	Which stack the team will choose (deferred, see Decision D-002).	Untestable/vague requirements block quality evidence at M3.
Deployment environment	Availability (NFR-004 candidate) and environment parity depend on where the system will run.	Hosting platform decision in M2.	Team's budget ceiling and familiarity with candidate platforms.	Late discovery of platform incompatibility disrupts the M3 release schedule.

Data backup/recovery	Even an MVP holds real (if simulated) service-request data that requesters rely on.	Persistence and backup design in M2/M3.	A concrete data-retention/recovery policy.	Data loss with no recovery path undermines the whole platform's credibility.
Observability	Management's dashboards need (FR-006) implies logging/monitoring must exist, not just a UI.	Logging/monitoring tool selection.	Which metrics matter most to management oversight.	No way to demonstrate system health or diagnose issues at M4.
Scalability	Expected concurrent user load is still unknown and affects NFR-001.	Architecture pattern choice in M2.	A realistic estimate of concurrent requesters/staff.	Performance NFR silently fails under real usage, discovered too late.
Maintainability	Three people with individually assessed accountability must all be able to read each other's code.	Coding standard/architecture pattern agreed in M2.	An agreed style guide/lint configuration.	Inconsistent code increases technical debt and slows M3 review.

Engineering Decision Log

ID	Context	Constraints	Alternatives	Decision	Rationale	Trade-offs	Risks	Evidence	Later consequence
D-001	Need version-control governance in place before any controlled artefact work begins.	Master Project Brief mandates a protected main branch and a minimum of two reviewer approvals.	(a) Single shared main branch, no protection, fast but ungoverned. (b) Protected main with mandatory 2-reviewer PRs.	Option (b) adopted.	Required by the Master Brief; enforces a real quality gate rather than a documentation exercise.	Slower merge velocity with only 3 available reviewers (author can never self-approve).	Risk of review becoming a bottleneck or a rubber stamp under deadline pressure (see R-004).	GitHub branch-protection settings screenshot + PR history.	Revisit at M3 once real PR volume is known.
D-002 (deferred)	Team must eventually choose a backend/frontend technology stack.	M1 explicitly excludes final technology-stack selection as a required decision.	Not yet meaningfully explored — no quality-attribute prioritisation or skills inventory has been done yet.	DEFERRED to Milestone 2.	Insufficient evidence currently exists to justify a defensible choice; deciding now would be a guess, not an engineering decision.	—	Risk R-001 (skills gap) is directly tied to whichever stack is eventually chosen.	—	To be decided once architecturally significant requirements and quality attributes are identified in M2.

GitHub and Team Governance

1. One controlled team repository, structured per the Master Brief's suggested layout (docs/PED, requirements, risk, decisions, architecture, change; src; tests; GitHub/workflows).
2. Main branch protected: direct pushes disabled; all substantive changes to controlled artefacts or code enter via Pull Request.

3. Minimum two approvals required per PR, from reviewers other than the author; self-approval disabled in repository settings.
4. Issues/board used to track engineering work (documentation and registers, not only application code) from the start of M1, so repository history shows authentic progressive participation from all three members.
5. No secrets, tokens or credentials committed to the repository; a .gitignore and environment-variable convention were agreed before any configuration work began.
6. GitHub Repo Access

Overview, Release Candidate & Current Status

PED v3.0 continues PED v2.1 — it does not replace it. M1 (requirements baseline) and M2 (architecture, technology and initial design baseline) content above is preserved. This part of the PED is an evidence index: each claim points to authentic repository, CI, test or register evidence rather than duplicating it.

Honest status statement (as at 16 Oct 2026): the release candidate is still being finalised. Where an item is incomplete, partially verified or not yet run, this PED says so explicitly and the Release Evidence Summary (M3.10) classifies readiness on that basis. A justified NOT READY or CONDITIONALLY READY conclusion is acceptable; an unsupported READY claim is not.

Implemented scope of the release candidate

FR	Requirement (from M1 baseline)	M2 module / component	M3 status
FR-001	Requester submits a service request	Request Management	[Implemented]
FR-002	Requester views current status	Request Management	[Partial / Not started]
FR-003	Staff view relevant requests (RBAC)	Staff Assignment + Identity & Access	[Partial / Not started]
FR-004	Staff assign / accept a request	Staff Assignment	[Partial / Not started]
FR-005	Controlled status transitions + audit entry	Request Management (Strategy rules) + Audit	[Partial / Not started]
FR-006	Staff comments / actions on a request	Staff Assignment	[Partial / Not started]
FR-007	Management view open / overdue / resolved / closed	Reporting	[Partial / Not started]

Where each M3 rubric area is evidenced

Rubric area (team, 30)	PED section	What is shown in the recorded video
A. Implementation & M2 baseline (5)	M3.1, M3.2, M3.3	Working integrated scope, release ID, API/persistence interactions, controlled deviations
B. Collaboration & configuration (4)	M3.4	Git history, feature branches, protected main, PRs with two independent reviews, secrets handling
C. Automated verification & CI (6)	M3.5, M3.7	Suite run, CI run, a check that blocks progression, interpretation of results
D. Black-box / integration / system (6)	M3.7	Technique choice + inputs, expected vs actual, API/integration, E2E, negative cases
E. Quality, defects & traceability (4)	M3.9, RTM section	RTM chain requirement → AC → implementation → test → evidence → defect/status; defects and debt
F. Performance & staging (3)	M3.8	One justified load exercise; staging deployment; limitations
G. Release review & residual risk (2)	M3.10	Release Evidence Summary, classification, residual risks

M2 Baseline Conformance

What the M2 baseline decided, and whether the implementation followed it. Only significant deviations require a change record (M3.3); minor coding-detail changes do not.

M2 decision (reference)	Implemented as planned?
Modular monolith, five modules: Identity & Access, Request Management, Staff Assignment, Reporting, Audit (ADR-A01/A02)	[Yes / Partly / No]
ASP.NET Core Web API + React (Vite/TS) + PostgreSQL, Option 2 (ADR-T01)	[Yes / Partly / No]
PostgreSQL relational schema; ACID transaction for status update + StatusHistory (ADR-D01, NFR-004)	[Yes / Partly / No]
Strategy pattern: one ITransitionRule per status change in StatusWorkflowService (Design Problem 1)	[Yes / Partly / No]
Repository + Unit of Work over EF Core (Design Problem 3)	[Yes / Partly / No]
Notification channel decoupling (Design Problem 2)	[Yes / Partly / No]
JWT authentication with role claims; RBAC at API layer (SE-004)	[Yes / Partly / No]
Deployment on free/low-cost tier; API + static SPA; CORS configured (CR-001)	[Yes / Partly / No]

Controlled Changes

Format for every significant change: original decision → implementation evidence/problem → revised decision → consequence. Do not hide a justified change simply to look consistent with M2.

Ref	Original decision	Problem / evidence	Revised decision	Consequence	ADR / PR
CC-001	PED v2.1 contained unresolved placeholders (ADR-D01 decision, ADR-T01 decision, R006/R007 wording, "TBD" test links)	Placeholders left baseline decisions ambiguous	Completed in PED v3.0 from the M2 content already in this document (no decision was changed)	Decision records are now self-contained; RTM test links point to test IDs	This PED version
CC-002	[M2 decision that changed during implementation]	[what went wrong / evidence]	[new decision]	[maintainability / testing / deployment impact]	[ADR-x / PR #]

Construction & Configuration Control

- Repository strategy (continued from the Governance section): one controlled repository, feature/fix branches, protected main, no direct pushes, minimum two approvals from reviewers other than the author, self-approval disabled.
- Progressive history: commits and PRs from all three members across M1–M3, not a final upload. [Link to commit graph / contributors page]
- Secrets: no credentials in the repository (SE-002). Configuration via environment variables / secret store; .gitignore agreed before configuration work. [Name the mechanism used, e.g. user-secrets, .env not committed, GitHub Actions secrets]
- Release identification: the exact tag/commit above is the version demonstrated; staging reports the same identifier. [How the deployed version is shown, e.g. /health or footer]

Evidence to show	Link / location
A PR with two independent approvals and review comments that led to a change	[PR #, what changed because of the review]

Branch-protection settings (required reviews + required status checks)	[settings screenshot or ruleset link]
Issue/task linked to a PR	[PR #, what changed because of the review]
Secrets handling (configuration file template without secrets; CI secrets)	[settings screenshot or ruleset link]

Known governance gap (if applicable): [e.g. any commit made directly to main, any PR merged with fewer than two reviews — state it and explain; risk R-004].

CI & Automated Verification

One working pipeline is sufficient. The table lists the checks the pipeline is intended to run; confirm each against the actual workflow file and replace the highlighted text with what really runs.

Stage	Check	Runs on	Blocks merge?
Build (API)	dotnet restore / build	PR + push	[Yes]
Automated tests	dotnet test — unit/component + API/integration suites	PR + push	[Yes]
Build (SPA)	npm ci, lint, vite build (TypeScript)	PR + push	[No]
Static / code quality	[analyser or linter actually configured]	PR + push	[Yes]
Dependency & secrets	[dotnet list package --vulnerable / npm audit / Dependabot / GitHub secret scanning — whichever is enabled]	PR + push	[Yes]

Quality gate: [state the rule, e.g. "PR cannot merge unless build and tests pass and two reviewers approve"]. Demonstrated blocking failure: [PR # / run link where a red check blocked progression — required by the rubric].

What a green run does and does not prove: a green run shows the configured checks passed on this commit. It does not prove that untested behaviour is correct, that coverage is meaningful, that the staging configuration matches production, or that performance targets are met. Coverage: [% if reported] — it shows which lines executed, not whether assertions are meaningful. Flaky tests are investigated and fixed, never simply re-run until green.

Minimum verification baseline — status

Category	Minimum	Cases designed (M3.7)	Executed and passing
Unit / component	5	6 (TC-U01–U06)	[ ]
Black-box functional	5	8 (TC-B01–B08)	[ ]
API / integration	3	4 (TC-I01–I04)	[ ]
E2E / system	2	2 (TC-E01–E02)	[ ]
Negative / failure / unauthorised	3	8 flagged NEG	[ ]
High-priority requirement / risk	5	10 flagged HP	[ ]
Black-box techniques	2	EP, BVA, state transition, decision table	[which are actually used]
Separate performance scenario	1	PT-01	[ ]

Only cases that are actually automated and executed count. If a designed case is not implemented by the freeze, remove it from the claim or mark it "Not run" — do not present it as passing.

Quality Strategy Applied (risk-based)

Verification effort was concentrated where failure would hurt most, rather than spread evenly or driven by test counts.

Risk / ASR	Why it ranks high	Stronger verification applied	Test IDs

ASR-001 / NFR-004 — atomic audit of every status change	Lost or partial audit entries break accountability, the core problem in the Problem Statement	Atomicity unit test with failing history write; API test counting audit rows after 5 transitions	TC-U04, TC-I02
SE-004 — authentication & RBAC (ASR-003/006)	Wrong access exposes requester data or lets requesters change status	Unauthorised/negative API tests; decision table of role × action	TC-U05, TC-B04, TC-B06, TC-I03
FR-005 — status transition correctness (Design Problem 1)	Core, frequently changed business rule	State-transition tests, valid and invalid	TC-U01–U03, TC-I04
R006 / ASR-002 — free-tier DB connection limit vs 50 users	Could silently fail the concurrency target	Small load test on dashboard/list endpoint against staging DB	PT-01
R-004 — two-review rule skipped under pressure	Quality gate becomes a rubber stamp	Technical branch protection + merge-history review before submission	M3.4

Black-Box, Integration & System Validation

Techniques were chosen because the behaviour requires them: state transition for the status workflow (history-dependent), equivalence partitioning and boundary value analysis for request-submission input, a decision table for role-based permissions (several conditions interact), plus negative/unauthorised testing. Expected results are derived from the acceptance criteria in this PED before execution. The cases below are the designed suite; the last column must record the real result and evidence link — "Not run" is an acceptable, honest status.

ID · level · technique	Basis	Input / precondition	Expected result	Flags	Status · evidence
TC-U01 · Unit · State transition	FR-005 / AC-FR-005	Request in Submitted; staff user; target Assigned	Transition permitted; status updated; new status and timestamp recorded	HP	[Pass/Fail/Not run · link]
TC-U02 · Unit · State transition (invalid)	FR-005, Design Problem 1	Request in Submitted; target Closed	Rejected with domain error; status unchanged	NEG HP	[Not run · link]
TC-U03 · Unit · State transition (terminal)	FR-005	Request in Closed; target In Progress	Rejected; Closed is terminal [confirm rule in code]	NEG	[Not run · link]
TC-U04 · Unit/component · Atomicity	NFR-004, ASR-001	Fake Unit of Work where the history insert throws	Status not changed and no partial write: both succeed or both fail	HP	[Not run · link]
TC-U05 · Unit · Authorisation	SE-004, FR-005	Requester role attempts a status change	Denied; no change	NEG HP	[Not run · link]
TC-U06 · Unit · EP	FR-001	One valid input set; sets missing each required field	Valid accepted; each invalid rejected with a validation error	NEG	[Not run · link]
TC-B01 · Black-box · EP	FR-001	Valid submission, one representative per valid class	Request created with a unique ID	HP	[Not run · link]
TC-B02 · Black-box · BVA	FR-001	Text field at min−1, min, max, max+1 [confirm real limits]	Inside limits accepted; outside rejected	NEG	[Not run · link]
TC-B03 · Black-box · Acceptance	FR-002 / AC-FR-002	Requester with two requests in different statuses	Requester sees own requests with current controlled status	HP	[Not run · link]
TC-B04 · Black-box · Unauthorised	FR-002, SE-004	Requester A opens Requester B's request	Denied (403/404); no data leaked	NEG HP	[Not run · link]
TC-B05 · Black-box · Acceptance	FR-004 / AC-FR-004	Staff assigns a request to themselves	Responsible person recorded on the request	HP	[Not run · link]
TC-B06 · Black-box · Decision table	SE-004, FR-003–007	Role (Requester/Staff/Management) × action (submit, assign,	Every combination matches the permission table [attach table]	HP	[Not run · link]

		change status, view dashboard)			
TC-B07 · Black-box · EP	FR-007	Seeded requests across open / overdue / resolved / closed	Filter/grouping returns the correct set for each		[Not run · link]
TC-B08 · Black-box · Acceptance	FR-006 / AC-FR-006	Staff adds a comment	Saved against the request with author and time		[Not run · link]
TC-I01 · API/integration	FR-001	POST new request with valid token (API → PostgreSQL)	201 Created; row persisted with unique ID [confirm route/code]	HP	[Not run · link]
TC-I02 · API/integration	NFR-004 / AC-NFR-004	Five status changes via API	Five audit rows, each with timestamp, user ID, previous and new status	HP	[Not run · link]
TC-I03 · API/integration · Negative	SE-004	Protected endpoint with no token; with wrong role	401; 403 respectively	NEG HP	[Not run · link]
TC-I04 · API/integration · Negative	FR-005	Invalid transition via API	4xx; database unchanged; no audit row	NEG	[Not run · link]
TC-E01 · E2E/system (staging)	FR-001/002/004/005	Requester submits → staff assigns → staff resolves → requester views	Requester sees Resolved; history shows each step	HP	[Not run · link]
TC-E02 · E2E/system (staging)	FR-007, NFR-004	After TC-E01, management opens dashboard	Counts reflect the new state; audit trail complete		[Not run · link]

Interpretation to state in the video: what these results demonstrate (the acceptance criteria are met for the covered behaviour) and what they do not (untested paths, data volumes, browsers, production configuration). A passing test only supports an acceptance criterion when it checks that criterion.

Performance, Staging & Reliability

Performance exercise PT-01

Field	Content
Operation and why	Dashboard / request-list load for staff and management. NFR-001 and NFR-002 make this the performance-sensitive journey, and R006 (free-tier connection limit) threatens it.
Workload	[tool: k6 / JMeter; virtual users (scaled-down from the NFR-002 target of 50 if needed); duration; actions per user]
Environment & data	[where run (staging vs local), hardware, database tier, number of seeded requests]
Target / baseline	From M1: dashboard < 2 s in 95% of runs (AC-NFR-001); under load average < 3 s, max < 5 s, all HTTP 200 (AC-NFR-002). If the workload differs from NFR-002, say so.
Measured result	[avg / p95 / max response time; success and error rate; throughput if available]
Interpretation	[met / not met against each target, and why]
What it does NOT prove	Municipal production scale, long-duration behaviour, real network conditions, or behaviour under a different database tier. It is a small student-capacity exercise.

Staging / test deployment

Field	Content
Type	[Real cloud staging OR credible simulated staging (e.g. separate configuration/profile + separate database) — no bonus marks for either]
Deployed candidate	[how the deployed version is shown to match the release tag]
Critical journey on staging	TC-E01 [run against staging: yes/no]
Differences from development	[configuration, database, TLS/HTTPS, secrets, data volume — and which could change production behaviour]

Rollback / recovery	Restore previous release tag [tag] and redeploy; consider database migrations (backward compatibility, data to preserve) and environment configuration/secrets that must be restored with it.

Known limitations of the original NFR measurement plans: AC-NFR-005 requires a 2-week uptime observation, which cannot be completed within M3 and is recorded as a limitation rather than a pass; AC-NFR-003 (SSL Labs rating) needs a publicly reachable HTTPS site, so with simulated/local staging it is [not verifiable / verified on cloud staging].

Defects, Findings & Quality Interpretation

Defect register (continued)

ID	Defect	Severity / impact	Linked req / test / risk	Status	Fix + regression evidence
D-101	[real defect found during testing]	[High]	[TOC05]	[Open/Fixed]	[PR / test]

If no defects were found, say so and explain why that is credible (what was tested); do not leave the register silently empty. Greatest current concern: [defect and why].

Static analysis & security/dependency findings

Tool	Finding	Severity / relevance	Handling (fix / mitigate / accept)
[ ]	[ ]	[ ]	[ ]

Technical debt register (continued)

ID	Deferred compromise / concern	Consequence if ignored	Planned action
TD-01	[e.g. Repository/UoW abstraction adds boilerplate over EF Core]	[Fail]	[soon to be implemented]
TD-02	NFR-006 accessibility scan (axe/Lighthouse) [done / partial / not done]	Level A violations undiscovered	[soon to be implemented]
TD-03	[known shortcut or unfinished feature]	[Fail]	[M4 / owner]

M3.10 Release Readiness & Residual Risk

Release Evidence Summary

Evidence area	Result	Interpretation	Concern / action
Build & CI	[Fail]	What the configured checks demonstrate	[block/fix if a mandatory gate fails]
Automated regression	[Fail]	Is critical behaviour protected?	[gaps]
API / integration	[Fail]	Are critical integrations behaving?	[gaps]
System / E2E	[Fail]	Does the critical journey complete on staging?	[missing acceptance]
Static / security	[Fail]	Severity and relevance of findings	[block risk]
Performance	[Fail]	Against NFR-001/002 targets and environment	[gaps]
Known defects	[Fail]	Business impact	[block or residual risk]
Acceptance evidence	[Fail]	Agreed behaviour (FR-001–007) demonstrated?	[missing acceptance evidence]

Classification: [READY / CONDITIONALLY READY / NOT READY]. Reason, grounded in several evidence sources above: [ ]. Rule: READY only if required evidence passes and no release-blocking issue is known; CONDITIONALLY READY if core evidence is credible with explicit non-blocking conditions; NOT READY if a gate, critical behaviour or blocker is unresolved. Because work is not finished, a justified CONDITIONALLY READY or NOT READY is expected to be the more honest outcome unless the evidence genuinely supports more.

Residual risks

Residual risk	Why it remains	Current mitigation	Next action / owner
Production-scale load not tested	Student staging cannot represent real usage; free-tier DB limits (R006)	Small load test PT-01 + architecture review	Re-test in a representative environment before real release / [owner]
Staging differs from production (TLS, config, data)	No production environment in scope for M3	Documented configuration differences	Verify in M4 deployment / [owner]
Availability (NFR-005) and TLS rating (NFR-003) not measurable within M3 timeframe	2-week observation and public HTTPS needed	Recorded as limitations, not passes	M4 / [owner]
[Incomplete or partially verified feature]	[ ]	[ ]	[ ]
AI-assisted code or tests accepted without full understanding	Time pressure near deadline	AI Usage Register; review by two other members	Every member re-explains own AI-assisted changes / Everyone

Do not keep any row that does not reflect a real remaining uncertainty.

Registers & Evidence Links (continued)

Risk register — M3 additions

ID	Description	Cause	Prob.	Impact	Mitigation	Owner	Status
R008	Core scope not fully integrated at the M3 freeze	Late integration; fixed deadline (SH-001)	High	High	Freeze scope; declare partial items in the Release Evidence Summary; no new features	Everyone	Open
R009	RTM and evidence assembled retrospectively rather than evolved	Time pressure	Medium	Medium	Update RTM rows in the same PR as the test; show repo history	Everyone	Open
R010	A member cannot appear or defend in the recorded video (0/50 for that student)	Equipment/access failure, unrehearsed content	Low	High	Test camera, audio and sharing beforehand; backup device; rehearse five defence areas	Everyone	Open

R006 and R007 (above) were reworded in PED v3.0 (see CC-001); R-001 to R-005 remain open pending the M3 review: [update status/probability for each based on what actually happened].

Linked controlled artefacts

Artefact	Location
RTM (continued)	[repo path / link]
Defect Register	[ ]
Risk Register	Above + [repo path]

ADR / decision records	[repo docs/decisions]
Technical Debt Register	Above + [repo path]
AI Usage Register	Above + [repo path]
CI workflow & runs	[ ]
Test reports	[ ]
Release Evidence Summary	M3.10 above

AI Usage Register

Date	Student	Tool	Engineering task	AI contribution	Verification	Decision	Issues found
3 Sep 2026	Katlego	Deepseek	Drafting initial FR-001–FR-007 and NFR006	Suggested first-pass phrasing and possible acceptance-criteria structure	Cross-checked each suggestion against the stakeholder analysis; rewrote vague phrasing to be testable	Accepted with edits	Initial AI draft used non-measurable language (“system should be fast”); replaced with numeric criteria
4 Sep 2026	Busisiwe	Claude	Drafting Stakeholder Analysis and Scope	Suggested generic stakeholder responses and in reach scope categories	Replaced generic entries with CivicConnect-specific scope tied to this team's actual constraints	Accpeted with edits	Generic AI-suggested scope were not specific enough on their own; rewritten with causes/owners
5 Sep 2026	Tshegofatso	Claude	Drafting the initial Risk Register and Forward Engineering Consideration	Suggested generic risk categories (schedule, technical, security, resourcing)	Replaced generic entries with CivicConnect-specific risks tied to this team's actual constraints	Accepted with edits	Generic AI-suggested risks were not specific enough on their own; rewritten with causes/owners
6 Sep 2026	Tshegofatso	Claude	Drafting PED document formatting	Suggested easy format derived from M1 document and Project document	Crossed-checked table formatting in M1 document and fact checked through different free-online PEDs	Accepted with edits	Initial AI draft used non-measurable language (“system should be fast”); replaced with numeric criteria

13 Sep 2026	Tshegofatso	Claude	Drafting intialASR and ADR				
14 Sep 2026	Katlego	Deepseek					

PED Baseline Sign-Off

Baseline Type	Milestone 1 — Engineering Foundation & Requirements Baseline (PED v1.0)
Version	V 2.0
Date	29 Sep 2026
Scope reviewed	YES
Requirements/traceability checked	YES
Risk review completed	YES
Repository/governance controls checked	YES
Outcome	ACCEPTED

Baseline type	M3 — Controlled Construction, Integration, Quality & Release Readiness
Version	PED v3.0
Date	16 Oct 2026
Release candidate	[tag / commit]
Scope reviewed	[YES]
Requirements/traceability checked	[NO]
Risk review completed	[NO]
Repository/governance controls checked	[NO]
Release classification	[NOT READY]
Outcome	[REVISION REQUIRED — set only after the checks above are genuinely complete]

Baseline type	M4 Final: Finalise, Consolidate & Conclude
Version	PED v4.0 Final
Final commit / tag	**CONFIRM**
Scope reviewed	**CONFIRM** YES / NO
Requirements / traceability checked	**CONFIRM** YES / NO
Risk review completed	**CONFIRM** YES / NO
Repository / governance controls checked	**CONFIRM** YES / NO
Deployment state	**CONFIRM** Production / Simulated production / Staging (no-go)
Release classification	**CONFIRM**
Outcome	**CONFIRM** ACCEPTED / CONDITIONALLY ACCEPTED / REVISION REQUIRED (set only after the checks above are genuinely complete)

Commented [TM8]: @Katlego Lethole Should we add the change logs?

Commented [KL9R8]: yep